



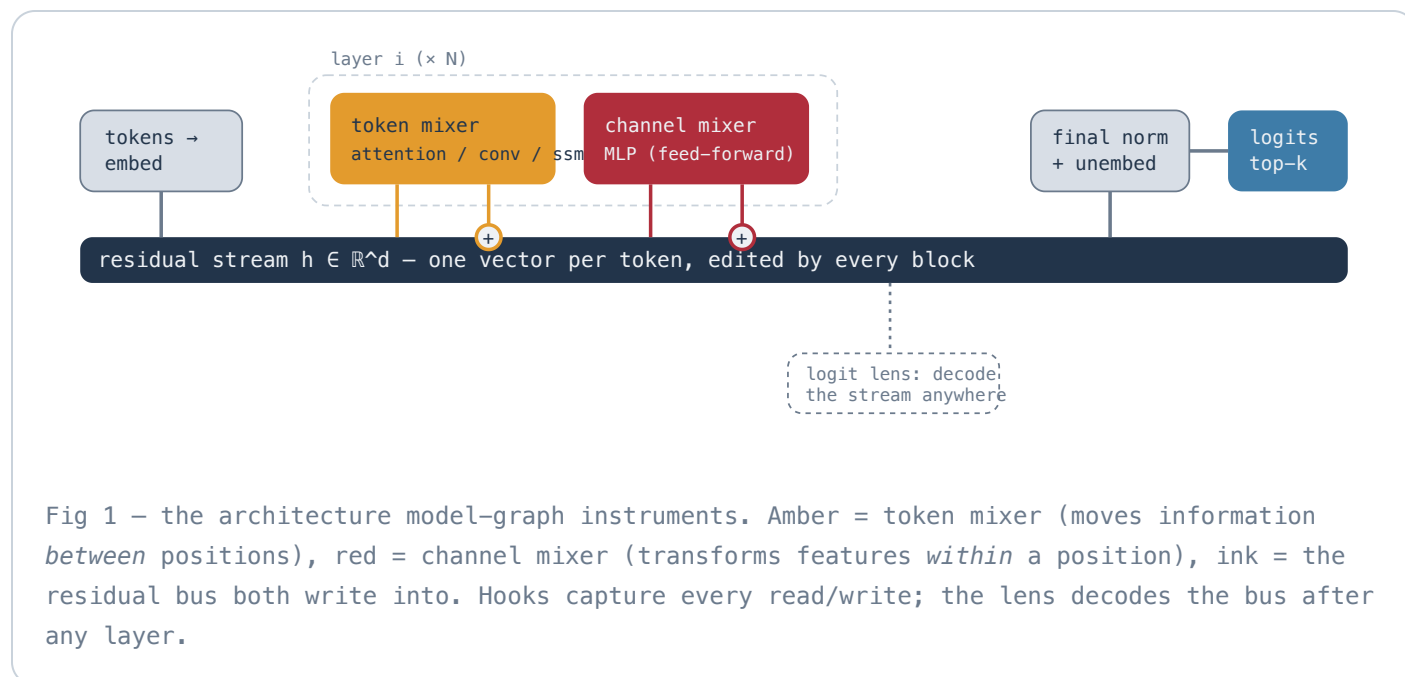
model-graph – the transformer, made visible

Product six-pager · July 2026 · model-graph.com · github.com/QSchlegel/model-graph · MIT

A transformer is not a black box – it is a very long assembly line whose every station writes into one shared workspace. model-graph instruments that assembly line with forward hooks and streams what each station does, token by token, into your browser. This document walks the architecture the way the tool visualizes it, with real captured examples at every step.

1 One shared workspace: the residual stream

Every modern LLM is organized around a single tensor per token position – the **residual stream**. Layers do not pass data to each other like a pipeline; each layer *reads* the stream, computes something, and *adds* its contribution back. The stream is a shared bus, and depth is a sequence of edits to it.



This is why model-graph's core view is a **layers × tokens heatmap of write norms**: how hard did each station edit the workspace, at each step? Rows whose writes suddenly dominate are where decisions happen. In the app: modes `mlp` / `attn` / `resid`, row-normalized because absolute norms grow with depth.

2 The channel mixer: MLPs as memory

Two thirds of a transformer's parameters sit in its MLPs. The useful mental model (Geva et al., 2021): each MLP is a huge **key→value memory** – the first projection asks "which of my stored patterns match this position?", the second writes the matching values back into the stream.

What the tool shows: in `mlp` mode, factual and content tokens fire large writes in the middle layers – you can watch a lookup happen. Function words barely disturb the stream. And the diagnostic that matters for real work: *a wrong fact emitted with a big mid-depth MLP write means the model retrieved a wrong memory, not that it failed to plan* – a genuinely different bug.

Example, captured live

Asking LFM2.5–1.2B "why is the sea salty?": the tokens `"minerals"` and `"dissolved"` ride hot MLP columns through L8–L13 while `"the"` and `"from"` stay pale – recall versus glue, visible at a glance.

Per-head and per-expert structure lives one click deeper: the cut-out panel decomposes any attention cell into 32 head norms, and any MoE cell into router flow (see §4).

3 Depth is compute: the logit lens

Because every layer only *edits* the shared stream, you can decode the stream after any layer through the model's own final norm and unembedding – as if the network stopped there. That is the **logit lens**. model-graph computes it for every token and marks every layer where the prediction *changes* with a dot; red where it flips to the token actually emitted – the **settle layer**.

A real ladder

L00	"ty"	100%	L08	"by"	0%
L01	" salt"	25%	L09	" Finger"	1%
L02	" "	7%	L10	" because"	79%
L03	" "	3%	L11	" because"	82%
L04	" salinity"	0%	L12	" because"	95%
L05	"Prot"	1%	L13	" because"	96%
L07	" undert"	1%	L15	" because"	100%

Fig 2 – logit lens for the token " because" (LFM2.5–1.2B, real capture). The topic surfaces early in whatever surface form is handy – " salt", then salt *in Chinese* – the middle layers churn, and the answer settles at L10 of 16. Highlighted rows are prediction flips; the red row is the settle layer.

Why it matters: settle depth is a per-token difficulty meter. Echoed and grammatical tokens settle in the first third; genuine choices stay contested. If the lens settles at L10/16, the last six layers mostly calibrated – the measured intuition behind early-exit and speculative decoding. Paired with per-token **entropy** (bits of uncertainty in the final distribution), you see *where in depth* and *when in time* the model made up its mind.

4 The token mixer: attention, heads, routing

Attention is the only place information moves *between* positions. For every generated token, model-graph captures where it looked (mean over heads, top sources) and how spread that attention was – drawn as **arcs over the token strip**.

◀ context 55% (mass on the prompt – 29% on the <|im_start|> sink)

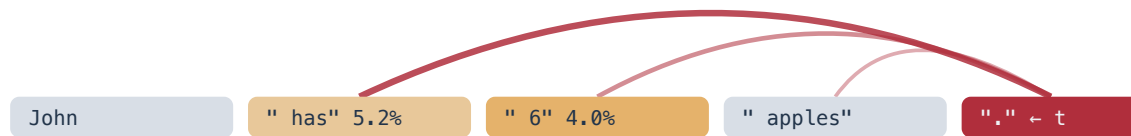


Fig 3 – attention sources for one token (L12, real capture from the apples arithmetic prompt). Arc weight = attention mass. Note the blue aggregate: over half the mass sits on the prompt – much of it parked on the first-token **attention sink**, not on the numbers.

Reading heads: in `attn` mode each attention cell subdivides into a per-head grid – the cell's *fingerprint*. Recurring stripes when text repeats are induction-style copying; fire-on-everything heads are usually sink-parking. Distinguishing "moved information" from "parked attention" is the core skill this view teaches.

When routing replaces density: MoE

router · E6		74%
router · E4		26%

For MoE layers the cut-out shows which experts the router chose per token and the routing-mass distribution across the whole run – expert collapse and load imbalance are visible as a lopsided histogram.

5 Beyond vanilla: hybrids, and the stream as a shape

Hybrid layer stacks



LFM2.5-1.2B: 10 gated short-conv layers (C, local mixing) + 6 global-attention layers (A, long-range retrieval)

Fig 4 – the hybrid stack model-graph runs by default. The efficiency race (conv/SSM/linear-attention hybrids) makes *which layer kind does what* a first-class question – compare A vs C rows in attn mode to watch the division of labour.

The stream as a shape: embedding trajectories

Each token's residual is a point in $\mathbb{R}^d$. Projected through a fixed random 3D basis (seeded – no refitting drift), generation traces a **path**: the prompt context as a blue dotted trace, generation walking out of it. The view auto-rotates and replays the generation step by step, endlessly.

- **Anisotropy**, visible: hidden states occupy a narrow cone, and special tokens (`<|im_start|>`) sit as huge-norm outliers – attention sinks in embedding form.
- **Loops** = the model cycling similar states (lists, repetition); **long jumps** = genuine semantic movement.
- Generation **orbiting the context cluster** reads as grounded; drifting far from it can be creativity – or hallucination.

All of this works serverless too: an observable-ONNX export declares every layer's residual as a graph output, so the in-browser WebGPU engine computes resid norms and trajectories from real tensors – validated against PyTorch hooks (argmax parity, norm drift within q4f16 noise).

6 The product around the physics

Zero-friction: one process speaks the OpenAI API – point any SDK or agent at it and every request is instrumented. Tool calls are parsed into spec-compliant `tool_calls`; vision-language models hook the language tower, so image chats are inspectable.

Testable: a curator records every run and gates CI with checks over internals (finite norms, depth trend, settle depth, uncertainty, attention focus) and tool-reasoning suites – should-call, abstain, argument extraction, result-grounded chains. Failures arrive with a run id and full internals attached.

Example of the loop closing: asked "*John has 3 apples, buys 2, eats 1*", the model answered 6 – and the capture shows why it's plausible: the answer token's attention parked 40% on the `<|im_start|>` sink instead of the numbers (Fig 3), and the lens shows "6" never seriously contested in depth. That failure is now a suite case; the suite is the regression gate.

Honest pixels, everywhere: nothing is rendered that wasn't measured. Missing signals say so on the canvas – per-head matrices, browser-run limits, all of it.

Open source: MIT, single repo, no telemetry. model-graph.com (live demo on a mock backend) · github.com/QSchlegel/model-graph – quickstart, contribution ladder and the docs vault (architecture & flow graphs) are in the repo. By Quirin Schlegel.